

OpenSeek-2 Count Nouns/Verbs 技术报告

1. 任务概述

本任务对应 OpenSeek-2 的 **Count Nouns/Verbs** 子任务，任务目标是在给定英文句子中统计名词或动词的数量。与普通语法题不同，本任务并不是简单按照传统英语语法规则进行名词、动词识别，而是要求尽可能匹配数据集自身的标注规则。

任务的输入通常是一条自然语言指令加一句英文图像描述，例如：

```
1 | Count the number of nouns in this sentence: 'A man holding a baseball bat'
```

或者：

```
1 | Count the number of verbs in this sentence: 'A man is using a cell phone to photograph a barn'
```

模型需要判断当前样例要求统计的是 nouns 还是 verbs，然后根据对应规则识别句子中应被计数的词项，最后输出数量。例如，在名词统计任务中：

```
1 | Sentence: 'A man holding a baseball bat'
```

按照数据集规则，`man`、`baseball`、`bat` 均可能被计入名词单位，因此预测数量为 `3`。而在动词统计任务中：

```
1 | Sentence: 'A man is using a cell phone to photograph a barn'
```

其中 `is` 作为助动词不计入，`using` 和 `photograph` 表示动作，应计入动词，因此预测数量为 `2`。

本代码的核心思路是：不直接让模型只输出一个数字，而是要求模型先进行逐词分析，输出被计入的名词或动词列表，再由程序统计该列表长度作为最终答案。这样可以提高预测过程的可解释性，也便于后续调试和错误分析。

2. 数据集结构与输入输出格式

代码通过 `task2_data_loader(file_path)` 读取 JSON 格式数据文件。数据文件主要包含以下字段：

```
1 | task_id
2 | task_name
3 | Definition
4 | examples
5 | test_samples
```

各字段含义如下：

字段	含义
<code>task_id</code>	当前任务 ID，例如 <code>openseek-2</code>
<code>task_name</code>	当前任务名称，例如 Count Nouns/Verbs
<code>Definition</code>	任务定义说明
<code>examples</code>	训练样例，包含输入句子和标准答案
<code>test_samples</code>	测试样例，只包含输入，需要生成预测结果

其中，`examples` 用于可选的训练集验证。每条样例通常包含：

```
1 {
2   "id": "sample_id",
3   "input": "Count the number of nouns in this sentence: '...'",
4   "output": ["5"]
5 }
```

`test_samples` 用于最终提交文件生成，每条测试样例通常包含：

```
1 {
2   "id": "test_sample_id",
3   "input": "Count the number of verbs in this sentence: '...'"
4 }
```

最终输出文件为 JSONL 格式，每行对应一个测试样例，格式为：

```
1 {"test_sample_id": "openseek-2-xxxx", "prediction": "5"}
```

其中：

- `test_sample_id` 来自测试样例中的 `id`；
- `prediction` 是程序预测出的名词或动词数量；
- 预测结果统一转换为字符串，符合提交格式要求。

3. 总体技术路线

本代码采用“模型逐词分析 + JSON 结构化解析 + 列表长度计数 + JSONL 输出”的整体流程。

完整流程如下：

```
1 读取任务 JSON 文件
2      ↓
3 获取 task_id / task_name / examples / test_samples
4      ↓
5 判断输入样例是 nouns 任务还是 verbs 任务
```

```
6      ↓
7  选择 NOUN_SYSTEM_PROMPT 或 VERB_SYSTEM_PROMPT
8      ↓
9  调用 Qwen3-4B 生成逐词分析 JSON
10     ↓
11 解析模型输出，提取 output 列表
12     ↓
13 用 output 列表长度作为预测数量
14     ↓
15 可选：在 examples 上验证准确率
16     ↓
17 对 test_samples 并行预测
18     ↓
19 保存 JSONL 提交文件
```

该流程的关键设计点在于：模型输出的不是最终数字，而是一个包含 `reason` 和 `output` 两个字段的 JSON 对象。例如：

```
1  {
2    "reason": "1. A->冠词，不计入；2. man->名词，表示可见人物，计入；3. holding->动词，现在分词表示动作，不按名词计入；4. a->冠词，不计入；5. baseball->名词，在 baseball bat 中按复合名词前项计入；6. bat->名词，表示可见物体，计入",
3    "output": ["man", "baseball", "bat"]
4  }
```

程序不会直接相信模型输出的数字，而是统计 `output` 数组长度。因此，只要模型能够正确列出应计入的名词或动词，程序就可以稳定生成最终数字。

4. 数据加载模块

代码中的数据加载函数如下：

```
1  def task2_data_loader(file_path):
2      with open(file_path, 'r', encoding='utf-8') as f:
3          data = json.load(f)
4      return (
5          data.get("task_id"),
6          data.get("task_name"),
7          data.get("Definition", []),
8          data.get("examples", []),
9          data.get("test_samples", [])
10     )
```

该函数主要完成以下工作：

1. 使用 `open` 打开本地 JSON 数据文件；
2. 使用 `json.load` 将文件内容解析为 Python 字典；
3. 分别读取 `task_id`、`task_name`、`Definition`、`examples` 和 `test_samples`；

4. 对缺失字段提供默认值，避免字段不存在时报错。

该模块是整个程序的入口。后续无论是训练集验证还是测试集预测，都依赖该函数提供的数据。

5. 模型调用模块

代码使用 OpenAI SDK 初始化了一个兼容 OpenAI API 格式的客户端：

```
1 client = OpenAI(  
2     api_key="dummy",  
3     base_url="https://flagos.io/flagos-lab/hw/node/HW-gpu57/port/22653/v1"  
4 )
```

模型调用函数为：

```
1 def qwen_api(messages, model="/Qwen3-4B/Qwen/Qwen3-4B", retries=3):  
2     for attempt in range(retries):  
3         try:  
4             res = client.chat.completions.create(  
5                 model=model,  
6                 messages=messages,  
7                 temperature=0.0,  
8             )  
9             return res.choices[0].message.content  
10        except Exception as e:  
11            if attempt == retries - 1:  
12                print(f"\nAPI 调用失败: {e}")  
13                return ""  
14            time.sleep(5)
```

该函数具有以下特点：

5.1 使用确定性生成参数

代码设置：

```
1 temperature=0.0
```

这表示模型输出尽可能稳定，减少随机性。对于分类、抽取和计数任务而言，稳定性比创造性更重要，因此低温度设置是合理的。

5.2 支持失败重试

函数设置：

```
1 retries=3
```

如果 API 调用失败，程序不会立刻终止，而是最多重试 3 次。每次失败后等待 5 秒再继续尝试。如果最后一次仍然失败，则返回空字符串。

5.3 输入采用 messages 格式

每次调用模型时，程序传入：

```
1 messages = [  
2     {"role": "system", "content": system_prompt},  
3     {"role": "user", "content": input_text}  
4 ]
```

其中 `system_prompt` 用于规定任务规则和输出格式，`input_text` 是当前样例的原始输入。

6. 系统提示词设计

本任务的核心难点在于，数据集的名词和动词计数规则并不完全等同于标准语法规则。因此，代码分别设计了两个较长的系统提示词：

```
1 NOUN_SYSTEM_PROMPT  
2 VERB_SYSTEM_PROMPT
```

程序会根据输入内容自动选择对应提示词。

7. 名词提示词设计

`NOUN_SYSTEM_PROMPT` 用于处理名词计数任务。它要求模型扮演一个 “dataset-aligned noun extractor”，也就是数据集对齐的名词抽取器。

该提示词明确说明：目标不是按照标准语法定义统计所有名词，而是按照图像描述数据集的标注习惯，统计句子中可见实体、物体、人物、动物、地点、场景部件等可数单位。

7.1 名词计数范围

提示词中将以下内容视为可计入名词：

类型	示例
人物	man, woman, boy, girl, child, people, person, player
动物	dog, horse, elephant, bird, giraffe, zebra
物体	bat, phone, chair, plate, suitcase, umbrella
地点/场景部件	room, street, court, kitchen, beach, field, wall, window

这种定义更接近图像标注任务中的“可见对象”或“场景元素”，而不是传统语法中的所有名词。

7.2 不计入名词的情况

提示词明确要求不统计以下类型：

类型	示例
代词或占位词	it, he, she, they, this, that, something
抽象词	idea, purpose, appearance, memory, life
图像描述元词	image, picture, photo, view, scene, background
纯形容词/颜色/状态	small, large, red, white, empty, open
纯方向或位置词	front, back, side, top, bottom, middle
表示动作的动词形式	walking, standing, holding, riding, playing

例如：

```
1 | A person holding an umbrella in front of a building
```

其中：

- person 计入；
- holding 是动作，不作为名词计入；
- umbrella 计入；
- front 在 in front of 中是方位结构，不计入；
- building 计入。

最终名词列表为：

```
1 | ["person", "umbrella", "building"]
```

7.3 复合名词规则

提示词对复合名词进行了专门说明。部分复合名词会被拆分计数，例如：

复合结构	输出
tennis court	["tennis", "court"]
baseball bat	["baseball", "bat"]
cell phone	["cell", "phone"]
living room	["living", "room"]
parking lot	["parking", "lot"]
fire hydrant	["fire", "hydrant"]
teddy bear	["teddy", "bear"]

但也有一些结构通常不拆分，例如：

结构	输出
motor bike	["bike"]
ski slope	["slope"]
swiss army knife	["knife"]
soft drink	["drink"]

这说明本任务的标注规则具有明显的数据集特定性，不能简单使用词性标注器或语法规则完成。

7.4 “of” 结构规则

提示词还规定了若干 of 结构通常需要同时计入前后两部分，例如：

短语	输出
group of people	["group", "people"]
bunch of bananas	["bunch", "bananas"]
couple of cars	["couple", "cars"]
pair of zebras	["pair", "zebras"]
piece of cake	["piece", "cake"]
slice of pizza	["slice", "pizza"]

这类规则同样体现出数据集对“可见单位”和“数量单位”的特殊处理方式。

8. 动词提示词设计

`VERB_SYSTEM_PROMPT` 用于处理动词计数任务。它要求模型扮演一个 “dataset-aligned verb extractor”，即数据集对齐的动词抽取器。

该提示词强调：动词统计也不是标准语法中的所有动词，而是统计描述动作、事件、过程或结果状态变化的词。

8.1 动词计数范围

提示词中将以下内容视为可计入动词：

类型	示例
动作动词	walk, hold, ride, sit, play, eat, catch, make, take
事件性 -ing 形式	walking, holding, riding, sitting, standing, grazing
结果/事件分词	painted, decorated, displayed, canned, parked, filled, shown, chopped

例如：

```
1 | Jars of food are being canned in boiling water
```

其中：

- `are` 是助动词，不计入；
- `being` 是助动词成分，不计入；
- `canned` 表示被处理的结果事件，计入；
- `boiling` 在此处修饰 `water`，不作为主要动词计入。

因此动词列表为：

```
1 | [ "canned" ]
```

8.2 不计入动词的情况

提示词明确要求不统计以下内容：

类型	示例
助动词	is, are, was, were, am, be, been, being
介词/小品词	on, in, at, with, near, from, to, into, over, under
名词性词语	jump, trick, game, rail, fun, shot
普通形容词/状态词	full, open, ready, bright, barefoot

例如：


```
1 | A skateboarder hitting a trick on a ramp
```

其中：

- `hitting` 是动作动词，计入；
- `trick` 是名词宾语，不计入动词；
- `on` 是介词，不计入。

最终动词列表为：

```
1 | [ "hitting" ]
```

8.3 上下文判断规则

动词提示词强调同一个词在不同上下文中可能有不同词性。例如：

```
1 | doing a jump
```

其中：

- `doing` 是动词，计入；
- `jump` 在这里是名词宾语，不计入动词。

而在其他语境中，`jump` 也可能作为动词。因此模型必须根据句子上下文逐词判断，而不能只依据词表匹配。

9. 输出格式约束

两个系统提示词都要求模型只输出 JSON 对象，且必须包含两个字段：

```
1 | {  
2 |   "reason": "逐词分析过程",  
3 |   "output": [ "word1", "word2" ]  
4 | }
```

其中：

- `reason` 是中文逐词分析过程；
- `output` 是被计入的名词或动词列表。

提示词还规定：

1. 只输出合法 JSON；
2. 不允许输出 Markdown；
3. 不允许输出 JSON 之外的解释；
4. `reason` 必须逐词分析；
5. `output` 必须是字符串数组；

6. 不能跳过句子中的词；
7. 不能只写简短总结。

这种格式约束有两个作用：

第一，提升输出可解析性。程序后续可以直接读取 `output` 字段并计算长度。

第二，提升模型推理稳定性。逐词分析要求模型显式判断每个词是否计入，减少漏判和误判。

10. 模型输出解析模块

由于大模型输出不一定完全稳定，代码设计了 `parse_json_output(raw_output)` 函数对模型输出进行多层解析。

该函数的解析策略分为三步。

10.1 直接 JSON 解析

程序首先尝试：

```
1 data = json.loads(raw_output)
```

如果模型输出本身就是合法 JSON，并且包含 `reason` 和 `output` 字段，则直接提取结果。

其中，`output` 字段会被清洗为字符串列表：

```
1 cleaned = [str(x).strip() for x in output if str(x).strip()]
```

这样可以去掉空字符串或格式异常的元素。

10.2 从文本中提取 JSON 块

如果直接 JSON 解析失败，程序会尝试用正则表达式从文本中抓取 JSON 块：

```
1 match = re.search(r'\{.*\}', raw_output, flags=re.S)
```

这种情况适用于模型输出了 JSON 之外的少量额外文字，例如：

```
1 下面是结果：
2 {"reason": "...", "output": ["man", "bat"]}
```

程序会提取 `{...}` 部分并再次尝试 `json.loads`。

10.3 兼容只返回列表的情况

如果前两步都失败，程序最后尝试：

```
1 items = ast.literal_eval(raw_output)
```

这用于兼容模型偶尔只返回列表的情况，例如：

```
1 ["man", "baseball", "bat"]
```

如果解析结果是列表，则程序仍然可以把它当成 `output` 使用，只是 `reason` 字段为空。

10.4 完全失败时返回空结果

如果所有解析方式都失败，函数返回：

```
1 {"reason": "", "output": []}
```

此时后续预测数量会变成 0。这种处理方式可以保证单条样例解析失败时不会中断整个测试集预测流程。

11. 训练样例处理模块

训练样例处理函数为：

```
1 def process_single_example(sample, noun_system_prompt, verb_system_prompt, task_id,
2   model_name):
3     sample_id = sample.get("id", "")
4     input_text = sample["input"]
5     gt = str(sample["output"][0]).strip()
6
7     lowered = input_text.lower()
8     if "count the number of nouns" in lowered:
9         system_prompt = noun_system_prompt
10        task_type = "nouns"
11    elif "count the number of verbs" in lowered:
12        system_prompt = verb_system_prompt
13        task_type = "verbs"
14    else:
15        system_prompt = noun_system_prompt
16        task_type = "unknown_default_nouns"
17
18    messages = [
19        {"role": "system", "content": system_prompt},
20        {"role": "user", "content": input_text}
21    ]
22
23    raw_output = qwen_api(messages, model=model_name)
24    parsed_items, parsed_reason = parse_output_items(raw_output)
25    pred = str(len(parsed_items))
26    is_correct = (pred == gt)
```

该函数主要完成以下步骤：

1. 读取样例 ID；

2. 读取输入句子；
3. 读取标准答案；
4. 判断当前任务是 nouns 还是 verbs；
5. 选择对应系统提示词；
6. 调用模型；
7. 解析模型输出；
8. 统计 `output` 列表长度作为预测值；
9. 与标准答案比较，得到是否正确。

返回结果中包含：

```
1 {
2     "task_id": task_id,
3     "sample_id": sample_id,
4     "input": input_text,
5     "task_type": task_type,
6     "gt": gt,
7     "model_output": pred,
8     "parsed_items": parsed_items,
9     "parsed_reason": parsed_reason,
10    "raw_output": raw_output,
11    "is_correct": is_correct
12 }
```

该结构非常适合调试，因为它不仅保存最终预测结果，还保存了模型原始输出、解析后的词列表和逐词分析理由。

12. 测试样例处理模块

测试样例处理函数为：

```
1 def process_single_test(sample, noun_system_prompt, verb_system_prompt, task_id,
2   model_name):
3     sample_id = sample.get("id", "")
4     input_text = sample["input"]
5
6     lowered = input_text.lower()
7     if "count the number of nouns" in lowered:
8         system_prompt = noun_system_prompt
9         task_type = "nouns"
10    elif "count the number of verbs" in lowered:
11        system_prompt = verb_system_prompt
12        task_type = "verbs"
13    else:
14        system_prompt = noun_system_prompt
15        task_type = "unknown_default_nouns"
```

```

16     messages = [
17         {"role": "system", "content": system_prompt},
18         {"role": "user", "content": input_text}
19     ]
20
21     raw_output = qwen_api(messages, model=model_name)
22     items, parsed_reason = parse_output_items(raw_output)
23     pred = len(items)

```

该函数与训练样例处理函数类似，但测试样例没有标准答案，因此不会计算 `is_correct`。它返回：

```

1  {
2      "task_id": task_id,
3      "sample_id": sample_id,
4      "input": input_text,
5      "task_type": task_type,
6      "model_output": pred,
7      "parsed_items": items,
8      "parsed_reason": parsed_reason,
9      "raw_output": raw_output
10 }

```

其中 `model_output` 是整数形式的预测数量，最终写入 JSONL 文件时会转换为字符串。

13. 任务类型判断机制

代码通过输入文本中的关键词判断当前任务类型：

```

1  lowered = input_text.lower()
2  if "count the number of nouns" in lowered:
3      system_prompt = noun_system_prompt
4      task_type = "nouns"
5  elif "count the number of verbs" in lowered:
6      system_prompt = verb_system_prompt
7      task_type = "verbs"
8  else:
9      system_prompt = noun_system_prompt
10     task_type = "unknown_default_nouns"

```

该设计简单直接，适用于当前数据集格式较固定的情况。

如果输入中包含：

```

1  count the number of nouns

```

则使用名词提示词。

如果输入中包含：

```
1 | count the number of verbs
```

则使用动词提示词。

如果两者都不包含，则默认按照名词任务处理，并标记为：

```
1 | unknown_default_nouns
```

该默认策略可以避免程序报错，但也存在一定风险。如果数据集中出现不规范输入，默认名词处理可能导致预测错误。因此后续可以考虑增加更严格的异常提示或任务类型检测逻辑。

14. 并行处理设计

代码使用 `ThreadPoolExecutor` 对 `examples` 或 `test_samples` 进行并行处理。主程序中设置：

```
1 | max_workers = 200
```

测试集预测阶段使用：

```
1 | with ThreadPoolExecutor(max_workers=max_workers) as executor:
2 |     for result in tqdm(executor.map(test_func, test_samples), total=test_total,
   |     desc="Predicting test"):
3 |         test_results.append(result)
```

并行处理的优势在于：

1. 每条样例之间相互独立；
2. 每条样例都需要调用模型接口；
3. API 调用通常存在等待时间；
4. 多线程可以提高整体吞吐量；
5. `tqdm` 可以实时显示处理进度。

对于本任务来说，单条样例的处理主要耗时来自模型 API 调用，而不是本地计算。因此多线程可以有效提高测试集预测效率。

不过，`max_workers=200` 也可能带来潜在问题。例如：

- 并发过高可能触发接口限流；
- 本地线程调度开销增加；
- 服务端负载过大时可能导致失败率提高。

因此，在实际运行中，可以根据接口稳定性将其调整为更保守的数值，例如 32、64 或 100。

15. 训练集验证流程

代码中保留了训练集验证逻辑，但当前被关闭：

```
1 | if False:
2 |     ...
```

如果将其改为：

```
1 | if True:
```

则程序会对 `examples` 中的所有训练样例进行并行预测，并统计准确率。

训练集验证流程如下：

```
1 | 读取 examples
2 |     ↓
3 | 并行调用 process_single_example
4 |     ↓
5 | 统计 correct_cnt
6 |     ↓
7 | 计算 accuracy
8 |     ↓
9 | 保存错误样本 wrong_cases
10 |    ↓
11 | 保存完整验证结果 JSON
```

验证结果中包含：

```
1 | eval_output_data = {
2 |     "task_id": task_id,
3 |     "task_name": task_name,
4 |     "evaluate_time": ...,
5 |     "model_name": model_name,
6 |     "prompt": NOUN_SYSTEM_PROMPT + VERB_SYSTEM_PROMPT,
7 |     "examples_total": total_cnt,
8 |     "examples_correct": correct_cnt,
9 |     "examples_accuracy": accuracy,
10 |    "wrong_cases": wrong_cases,
11 |    "all_results": eval_results
12 | }
```

该结构能够支持后续详细分析：

- 哪些样例预测错误；
- 错误样例属于名词任务还是动词任务；
- 模型输出了哪些词；
- 模型逐词分析理由是什么；
- 是模型判断错了，还是解析失败。

这部分虽然当前没有启用，但对于 prompt 调优非常重要。通过分析 `wrong_cases`，可以不断补充系统提示词中的规则和 few-shot 示例，从而提高最终测试集表现。

16. 测试集预测流程

当前代码中测试集预测逻辑处于开启状态：

```
1 if True:
2     ...
```

测试集预测流程如下：

```
1 读取 test_samples
2      ↓
3 构造 test_func
4      ↓
5 多线程调用 process_single_test
6      ↓
7 得到 test_results
8      ↓
9 转换为提交格式 output_data
10     ↓
11 写入 JSONL 文件
```

程序将每条测试结果转换为：

```
1 {
2     "test_sample_id": item["sample_id"],
3     "prediction": str(item["model_output"])
4 }
```

然后逐行写入目标文件：

```
1 with open(test_jsonl_path, 'w', encoding='utf-8') as f:
2     for item in output_data:
3         line = json.dumps(item, ensure_ascii=False)
4         f.write(line + '\n')
```

最终生成的 JSONL 示例为：

```
1 {"test_sample_id": "openseek-2-f907968b5d6340e8affa8452ea2b6f94", "prediction": "5"}
```

该格式符合常见比赛提交要求，也便于评测脚本逐行读取。

17. 异常处理与鲁棒性设计

本代码在多个位置加入了容错机制。

17.1 API 调用容错

`qwen_api` 支持最多 3 次重试。如果接口暂时失败，程序会等待 5 秒后再次尝试。最终失败时返回空字符串，而不是直接抛出异常终止程序。

17.2 JSON 解析容错

`parse_json_output` 支持三种解析方式：

1. 直接 JSON 解析；
2. 正则提取 JSON 块后解析；
3. 使用 `ast.literal_eval` 兼容列表输出。

这使得程序可以处理模型输出中的轻微格式偏差。

17.3 空输出容错

如果模型没有返回有效内容，解析函数返回：

```
1 | {"reason": "", "output": []}
```

此时预测数量为 0。虽然这不一定正确，但可以保证整个测试集预测流程不中断。

17.4 未知任务类型默认处理

当输入中无法识别 nouns 或 verbs 时，程序默认使用名词提示词：

```
1 | task_type = "unknown_default_nouns"
```

这可以避免异常，但也应在后续日志中重点检查。